

CMSC 451: Design and analysis of computer algorithms

Section 0201, Fall 2024

An algorithm is a well-defined computational procedure for processing some input to obtain a desired output.

This course: how to

- design algorithms
- analyze their performance

Why study algorithms?

- coding interviews
- they arise in software; may want to make it efficient even on large inputs
- solve a one-off problem
- algorithmic ideas are useful in other areas of science/technology
- interesting math.

Course structure

I. Introduction

- Example: stable matching problem
- Math background
- Basic graph algorithms

II. Algorithm design techniques

- Greedy algorithms
- Divide-and-conquer
- Dynamic programming
- Augmenting paths

III. Intractability

- NP-completeness
- Approximation algorithms

IV. Other models of computation

- Randomized algorithms
- Quantum algorithms

Algorithms are mathematical objects

Specifying an algorithm:

- pseudocode
- plain language description

<u>Ex:</u>	<pre>int foo = 0 for (i = 0; i ≤ 100; i += 2) if x[i] != 0 foo++ endif endif</pre>	<u>better</u> → let foo be a counter initialized to 0 for every even value of i from 0 to 100 if the ith entry of x is nonzero, increment foo.
------------	--	---

But don't oversimplify!

Ex: Repeatedly find adjacent elements that are out of order and swap them until the list is sorted.

Stable matching problem

Goal: Pair members of two sets according to their preferences

Ex: • students & employers for summer internships
• men and women to be married

for simplicity: the two sets have the same size

we must match each member of the first set with exactly one member of the second set

Def: Given two sets S, T , a matching is a set of pairs of the form (s, t) , $s \in S, t \in T$ such that every element of S appears in at most one pair, and similarly for T .

A matching is perfect if every element of S and T is in some pair.

Each member ranks all elements of the other set
(no ties; no omissions)

Def: A matching is stable if there is no pair (s, t) not in the matching such that s prefers t over its assigned partner and t prefers s over its assigned partner.

- Sign up for Piazza
- Today: Stable matching

Aug. 29

Ex: students A, B companies X, Y

preferences: $A: X, Y$ $X: A, B$
 $B: Y, X$ $Y: B, A$

stable matching: $(A, X), (B, Y)$ $\leftarrow$

~~(A, Y)~~ ~~(B, X)~~

Ex: preferences $A: X, Y$ $X: A, B$
 $B: X, Y$ $Y: A, B$

stable: $(A, X), (B, Y)$

~~(B, X)~~ ~~(A, Y)~~

$\{(B, X), (A, Y)\} \rightarrow (A, X)$ is an instability

Ex: A: X, Y X: B, A
 B: Y, X Y: A, B

Ex: ^{Students}
 A, B, C ^{companies}
 X, Y, Z

prefs: A: X, Y, Z X: B, C, A
 B: X, Z, Y Y: C, A, B
 C: Y, Z, X Z: A, C, B

stable? (A, X), (B, Z), (C, Y)

no: (B, X) would rather be together

Gale-Shapley algorithm

All companies and students are unengaged

While there is a company c that is unengaged and has not made an offer to every student

Let s be the highest-ranked student for c to whom c has not yet made an offer

If s is unengaged

(c, s) become engaged

Else

let c' be the company to which s is engaged
 if s prefers c to c' then
 (c, s) become engaged
 c' becomes unengaged
 endif
 endwhile
 Return the set of engaged pairs

Lemma 1. Once a student receives an offer, they remain engaged for the rest of the algorithm, and the quality of their engagement (according to their preferences) can only increase.

Proof. An unengaged student who receives an offer always becomes engaged.

An engaged student can only switch engagements; they can never become unengaged.

A student only switches if they prefer the new company. □

Lemma 2. As the algorithm proceeds, the sequence of students engaged to a given company can only decrease in preference.

Proof: Companies make offers in order of decreasing preference. $\square$

Lemma 3. If there are n companies (& n students) then the algorithm terminates after at most n^2 iterations of the while loop

Proof: Let $P(t)$ be the set of pairs (c, s) such that c has made an offer to s by the t^{th} iteration.

There's a new offer in every iteration, so

$$|P(t+1)| = |P(t)| + 1.$$

Since there are n^2 possible pairs, $|P(0)| = 0$, and $|P(t)|$ increases by 1 at each step, there can be at most n^2 steps. $\square$

- Assignment 1 available on Canvas. Due Sep 18. Sep 3
- Stable matching
- Math background: Asymptotic notation, running time analysis, graphs

Lemma 4. The G-S algorithm outputs a perfect matching.

Proof. New engagements are only created between parties that were not previously engaged or who broke a previous engagement to create a new one. Thus the set of engaged pairs is always a matching.

Suppose that at the end of the algorithm (which occurs by Lemma 3), there is some company c and student s who are not engaged. At some point, c must have made an offer to s (by the termination condition of the loop). By Lemma 1, s will be engaged at the end of the algorithm, a contradiction. $\square$

Theorem. The matching output by the G-S algorithm is stable.

Proof. Suppose there is some pair (c, s) such that c prefers s to its intern s' and s prefers c to their company c' .

Then c must have made an offer to s before s' . But c ended up with s' . What went wrong?

Either:

- s had an offer they preferred to c , so they rejected the offer
- s accepted the offer from c but later switched to an even better company.

But s ends up with c' , which is lower on their preference list than c . This is a contradiction. $\square$

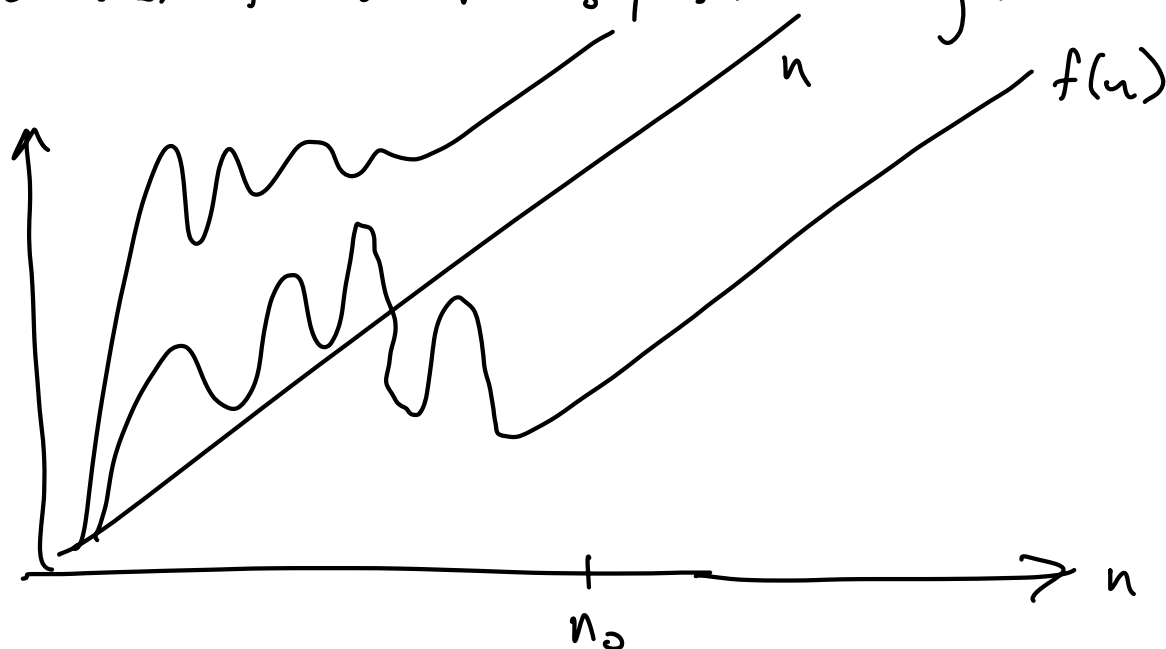
Asymptotic notation

We're interested in asymptotic upper bounds on the worst-case running times of algorithms.

We can express this using "big-O" notation.

Ex: running time $10n^2 + 2n + 5000$ is in $O(n^2)$

Def: We say $f(n) \in O(g(n))$ if $\exists c > 0, n_0 \geq 0$ such that for all $n \geq n_0$, $f(n) \leq c \cdot g(n)$.



We'll think of algorithms as efficient if they run in time polynomial in the input size n , i.e., $O(n^a)$ for some constant a .

Def: We say $f(n) \in \Omega(g(n))$ if $\exists c > 0, n_0 \geq 0$ such that for all $n \geq n_0$, $f(n) \geq c \cdot g(n)$.

Ex: $10n^2 \in \Theta(n^2), \Theta(n^3), \dots$
 $\in \Omega(n^2), \Omega(n), \dots$

Def: We say $f(n) \in \Theta(g(n))$ if $f(n) \in O(g(n))$
and $f(n) \in \Omega(g(n))$.

$$10n^2 \in \Theta(n^2)$$

Def: We say $f(n) \in o(g(n))$ if $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$
 $f(n) \in \omega(g(n))$ if $\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = 0$.

$$n^2 \in o(n^3), \quad n^2 \in \Theta(n^3)$$

Analyzing running times

- computing sums (iterative algorithm)
- solving recurrences (recursive algorithm)

Some common running times:

- logarithmic time: $\Theta(\log n)$
- linear time: $\Theta(n)$

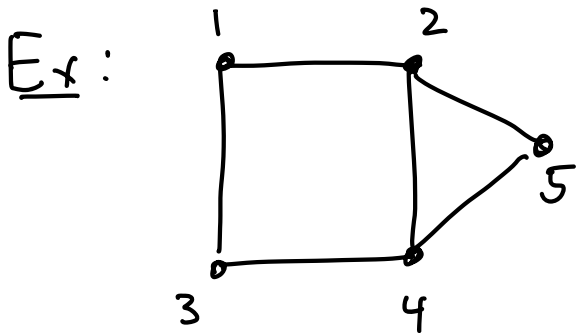
- tree $O(n^k)$ for some constant k
 $\binom{n}{k} = O(n^k)$
- exponential tree: $O(2^n)$
 $3^n \neq O(2^n)$

$$\lim_{n \rightarrow \infty} \frac{2^n}{3^n} = \lim_{n \rightarrow \infty} \left(\frac{2}{3}\right)^n = 0$$

$$2^{O(n)}$$

Graphs

Def: An undirected graph $G = (V, E)$ is a finite set V of vertices and E of edges, where edges are unordered pairs of vertices.



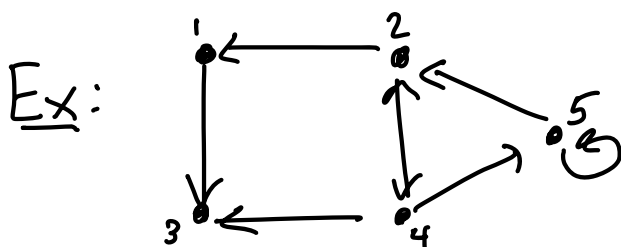
$$V = \{1, 2, 3, 4, 5\}$$

$$E = \{\{1, 2\}, \{1, 3\}, \{2, 4\}, \{2, 5\}, \{3, 4\}, \{4, 5\}\}$$

- Math background - graphs

Sep 5

- Graph algorithms: connectivity, BFS, bipartiteness



$$V = \{1, 2, 3, 4, 5\}$$

$$E = \{(1, 3), (2, 1), (2, 4), (4, 2), \{5, 5\}, \dots\}$$

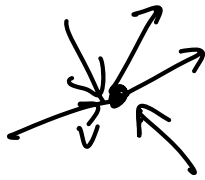
Terminology:

- vertices are sometimes called nodes
- edge $\{u, v\}$ joins u & v , which are its ends;
 u and v are adjacent; the edge is incident on u & v .
- edge (u, v) has tail u and head v .



- the degree of vertex v , $\deg(v)$, is the # of incident edges.

in a digraph, $\text{indeg}(v)$ is the # of edges with v as head
 $\text{outdeg}(v)$ is the # of edges with v as tail



$$\begin{aligned}\text{outdeg}(v) &= 2 \\ \text{indeg}(v) &= 3\end{aligned}$$

- we often write $|V| = n$, $|E| = m$
- $\sum_{v \in V} \deg(v) = 2m$

$$\sum_{v \in V} \text{indeg}(v) = m = \sum_{v \in V} \text{outdeg}(v)$$

- $m \leq \binom{n}{2} = O(n^2)$ in an undirected graph
 $m \leq n^2$ in a digraph

- a subgraph of $G = (V, E)$ is a graph $G' = (V', E')$
with $V' \subseteq V$ and $E' \subseteq E$

Graph representations

- Adjacency matrix: $|V| \times |V|$ matrix indexed by vertices, with $A_{uv} = \begin{cases} 1 & \text{if } (u,v) \in E \\ 0 & \text{otherwise} \end{cases}$
- Adjacency list: for each $v \in V$, given a list Adj_v of its v neighbors outgoing

Paths & cycles

A path is a sequence of vertices $(v_0, v_1, v_2, \dots, v_k)$ such that $(v_i, v_{i+1}) \in E$ for all $i \in \{0, 1, \dots, k-1\}$.

A path is simple if all its vertices & edges are distinct. The length of a path is k , its # of edges.

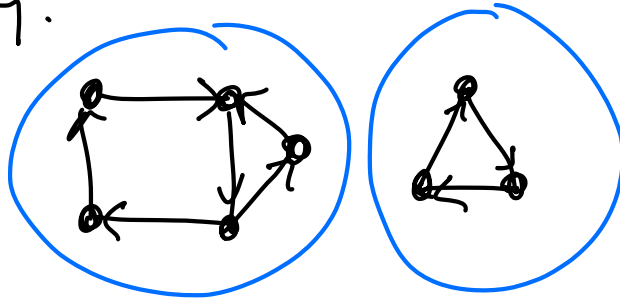
A cycle is a path with $v_0 = v_k$ and all other vertices distinct.

The distance between $u, v \in V$ is the length of a shortest path from u to v .

Connectivity

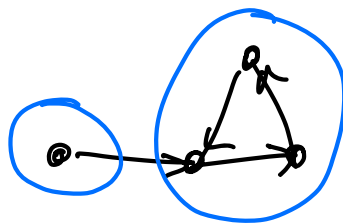
An undirected graph is connected if for all $u, v \in V$, $\exists$ a path from u to v .

A component of a graph is a set of vertices such that all pairs of vertices in the set have a path between them, and no superset has this property.



A digraph is strongly connected if $\forall u, v \in V$,
 $\exists$ a path from u to v and a path from v to u .

A strongly connected component is a maximal subgraph that is strongly connected.



Trees

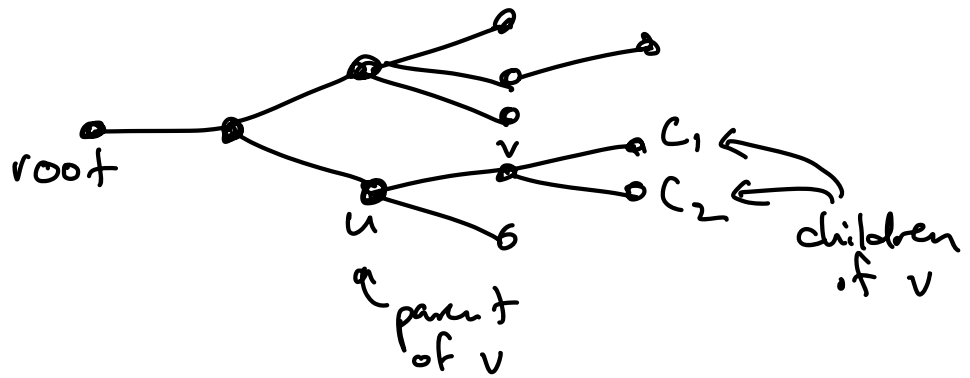
A forest is a graph with no cycles.

A tree is a connected forest.

A n -vertex tree has $n-1$ edges.

If we designate one vertex of a tree as the root, then the (unique) neighbor of a vertex

that is closer to the root is its parent, and the neighbors that are further from the root are its children.



Vertices of degree 1 in a tree are called leaves.

A digraph with no cycles is called a directed acyclic graph or DAG.

Connectivity

Given a graph, is it connected?

Given vertices s, t , is there a path between them?
($s-t$ connectivity)

One approach: find a spanning tree, a subgraph that includes all vertices and is a tree.

We can specify a rooted tree by giving the parent $pr[v]$ of every vertex v , with $pr[root] = \emptyset$.

Generic spanning tree algorithm (for a tree rooted at s):

Let T be the graph with one vertex, s , with $pr[s] = \emptyset$.

While $\exists$ an edge $\{u, v\}$ joining a vertex u of T
to a vertex v not in T

 Add vertex v and edge $\{u, v\}$ to T

 Set $pr[v] = u$

The resulting graph is always a tree (exercise)

Lemma: This algorithm produces a spanning tree
for the component of s .

Proof: Any vertex in the tree has a path to s :
 repeatedly take parents until you reach s .
To find a path from u to v in T ,
 join the paths from u to s and v to s .
On the other hand, if u is not in T ,
 then there is no path to s . If there were,
 we could follow the path and find an
 edge from a vertex not in T to a

vertex in T , but no such edge remains when the algorithm is done. $\square$

- Graph algorithms
 - Breadth-first search
 - Bipartiteness
 - Connectivity in directed graphs
 - Topological sorting

Sep 10

Breadth-first search

BFS: explore the graph in order of increasing distance from the root

Say a vertex is exhausted if it is not adjacent to any vertex outside the tree.

BFS(s):

Let T be the tree with one vertex, s , with $pr[s] = \emptyset$

Repeat:

Let u be the unexhausted vertex that joined the tree earliest (the "active vertex")

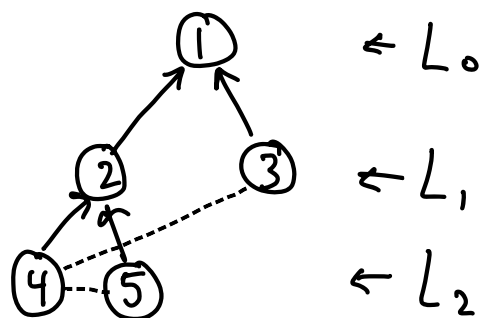
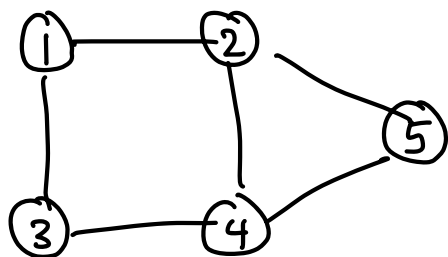
Choose an edge $\{u, v\}$ where v is not in T

Add vertex v and edge $\{u, v\}$ to T

Set $pr[v] = u$

Until all vertices are exhausted

Ex: BFS(1) on



Computing distances / shortest paths

BFS partitions the vertices into layers L_j

$$L_0 = \{s\}$$

L_{j+1} = neighbors of vertices in L_j that are not also neighbors of vertices in $L_0 \cup L_1 \cup \dots \cup L_{j-1}$.

Lemma: A vertex in L_j has distance j from s .

Proof: By induction on j .

Clearly $L_0 = \{s\}$ is the set of vertices at distance 0 from s .

If $L_0, L_1, \dots, L_j$ satisfy the lemma, then vertices in L_{j+1} must have distance at least $j+1$ from s .

But $\exists$ a path of length $j+1$ from any vertex in L_{j+1} to s since there is an edge to a vertex in L_j , which has a path of length j to s (by IH).

Hence vertices in L_{j+1} satisfy the lemma, so the lemma follows by induction. $\square$

Running time? (in terms of $n = |V|$, $m = |E|$)

Loop over all $u \in V$, and for each, do $O(\deg(u) + 1)$ operations for each

$$\begin{aligned} \text{Total running time: } & O\left(\sum_{u \in V} (\deg(u) + 1)\right) \\ & = O(m + n) \end{aligned}$$

Bipartiteness

Lemma: Let T be a BFS tree ^{of graph G} , and let u, v be vertices of T with $u \in L_i$ and $v \in L_j$. Suppose $\{u, v\}$ is an edge of G . Then $|i - j| \leq 1$.

Proof: Suppose u is added before v (WLOG). Then $i \leq j$ and u is active before v . Two cases:

① v is in the tree when u becomes active.

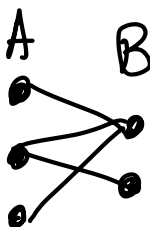
Then $\{u, v\}$ is a non-tree edge, and $\text{pr}(v)$ joined the tree before u , so $j = \text{layer}(\text{pr}(v)) + 1 \leq i + 1$

② v is not in the tree when u becomes active.

Then $\{u, v\}$ is a tree edge and $j = i + 1$. □

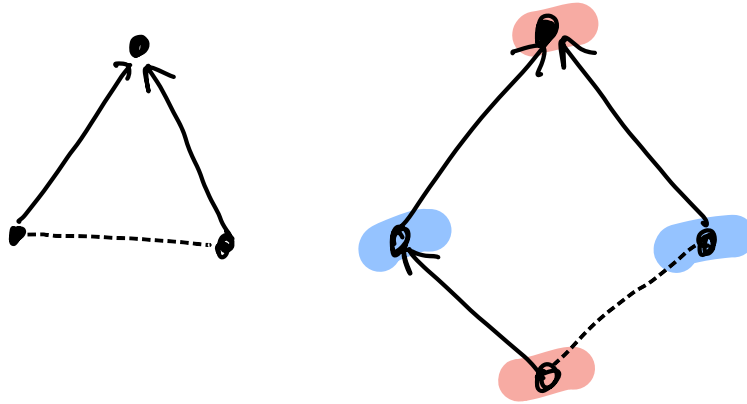
Def. A graph $G = (V, E)$ is bipartite if there is a partition (A, B) of V ($V = A \cup B$, $A \cap B = \emptyset$)

such that for all $\{u, v\} \in E$, either $u \in A$ and $v \in B$ or $u \in B$ and $v \in A$.



Theorem: A connected graph with BFS tree T is bipartite iff there is no non-tree edge joining vertices in the same layer.

Ex:



Proof: Suppose there is no non-tree edge joining vertices in the same layer.

Let $A = L_0 \cup L_2 \cup \dots \cup L_{2k}$ (vertices in even layers)
 $B = L_1 \cup L_3 \cup \dots \cup L_{2k+1}$ (vertices in odd layers)

This is a bipartition since all non-tree edges join vertices whose layers differ by 1.

Conversely, suppose $\{u, v\}$ is a non-tree edge between vertices in the same layer.

Suppose their nearest common ancestor is j layers higher.

Then the path in the tree between u and v has length $2j$.

Adding $\{u, v\}$ gives a cycle of length $2j+1$.
But an odd cycle cannot be bipartite. $\square$

- Graph algorithms
 - Connectivity in directed graphs
 - Topological sorting
- Greedy algorithms

Sep 12

Connectivity in digraphs

Can use BFS to find the vertices that can be reached from a given starting vertex.

Run BFS from every vertex (n runs of BFS)
Digraph is strongly connected iff all runs reach all vertices.

Running time: $O((m+n)n)$

Lemma: If u & v are mutually reachable,
and v & w are mutually reachable,
then u & w are mutually reachable.

Proof: Can go from $u \rightarrow w$ by going from $u \rightarrow v \rightarrow w$.
Can go from $w \rightarrow u$ by going from $w \rightarrow v \rightarrow u$.
□

Algorithm:

- Run BFS starting from s .
- Run BFS starting from s , reversing the directions of all edges.
- If both searches find all vertices, the graph is strongly connected; otherwise not.

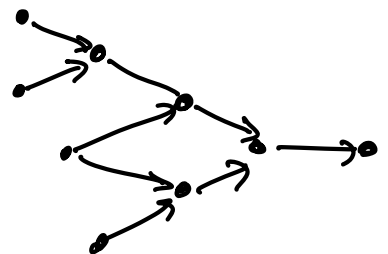
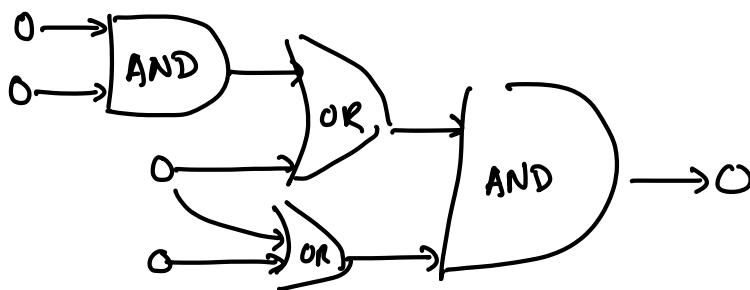
□

Running time: $O(m+n)$

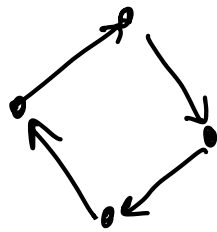
Topological sorting

Can use a digraph to capture a set of operations with constraints on which ones must be done before others.

Ex: Boolean circuits



A topological ordering is a possible sequence in which the actions can be performed, subject to the constraints on which other actions must have been completed.



← no topological ordering

Lemma: If G has a topological ordering, then G is a DAG.

Proof: For a contradiction, suppose G has a topological ordering $v_1, v_2, \dots, v_n$ and a directed cycle C .

Let v_i be the lowest-index vertex on C ; let v_j be the vertex before it on C .

Then (v_j, v_i) is an edge with $i < j$,

but we also must have $j < i$ for this to be a topological ordering. This is a contradiction. $\square$



Lemma. Every DAG has a vertex with indegree 0.

Proof: Prove the contrapositive: if every vertex has positive indegree then the graph has a directed cycle.

Start at any vertex. Walk along some edge in the backward direction (which is possible because every vertex has positive indegree). Repeat.

After at most $n+1$ steps, we must have visited some vertex twice, so we found a cycle. $\square$

Lemma. Every DAG has a topological ordering.

Proof: By induction on $n = |V|$.

A 1-vertex graph has a topological ordering.

IA: Suppose the lemma holds for any digraph with at most n vertices.

For an $(n+1)$ -vertex graph, let v be a vertex with indegree 0.

Put it first in the ordering.

Follow it by a topological ordering of $G \setminus \{v\}$.

This exists because $G \setminus \{v\}$ is obtained by deleting a vertex & edges from a DAG; deletion cannot create a cycle. (Also by IH.) $\square$

Theorem: A graph is a DAG iff it has a topological ordering.

Topo Sort (G):

let S be an empty set

for all vertices v

if v has indegree 0, add v to S

set $\text{count}[v] = \text{indeg}(v)$

endfor

while S is nonempty do

remove a vertex v from S

output v

for each vertex u that v points to

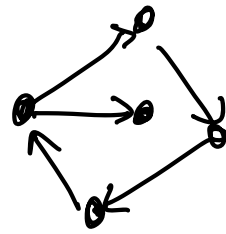
remove edge (v, u) from the graph

decrement $\text{count}[u]$

if $\text{count}[u] = 0$ then add u to S

endfor

end while



Runtime: $O(n)$ initialization

while loop goes through all vertices;

for loop for v takes $O(\text{outdeg}(v) + 1)$ steps

$$\Rightarrow O\left(n + \sum_v (\text{outdeg}(v) + 1)\right) = O(m + n)$$

- Assignment 1 due tomorrow (firm deadline!) Sep 17
- Greedy algorithms
 - Making change
 - Shortest paths
 - Minimum spanning trees

Ex: Making change

Fix coin denominations $c_1, c_2, \dots, c_k$

Given a value v , find a way to make up v
using as few coins as possible.

I.e., write $v = c_{i_1} + c_{i_2} + \dots + c_{i_n}$ with n as
small as possible.

Change (v):

if $v=0$, return.

else

let c be the largest coin of denomination at most v

add c to the list of coins

Change ($v-c$)

endif

Does this find the minimum # of coins?

Ex: • only coin 1 $\rightarrow$ optimal

• coins 1, 2 $\rightarrow$ optimal

• coins 1, 7, 10 ; $v = 14$

algorithm: $14 = 10 + 1 + 1 + 1 + 1$

but $14 = 7 + 7$

Shortest paths

Given weights $w_{uv} \geq 0$ for each edge $(u,v) \in E$,
what is the length of a shortest path from s to t
($s, t \in V$)?



Dijkstra's algorithm

Dijkstra (G, w, s) :

let $d[s] = 0$ and $pr[s] = \emptyset$

for all $v \in V \setminus \{s\}$, let $d[v] = \infty$

mark all vertices as unexplored

repeat:

let u be the unexplored vertex with smallest $d[u]$

for each neighbor v of u

if $d[u] + w_{uv} < d[v]$ then

let $d[v] = d[u] + w_{uv}$

let $pr[v] = u$

endif

endfor

mark u as explored

until all vertices are explored

Lemma: When v is explored, $d[v]$ is the distance from s to v .

Proof: By induction on the # of explored vertices.

Clearly holds when s is explored.

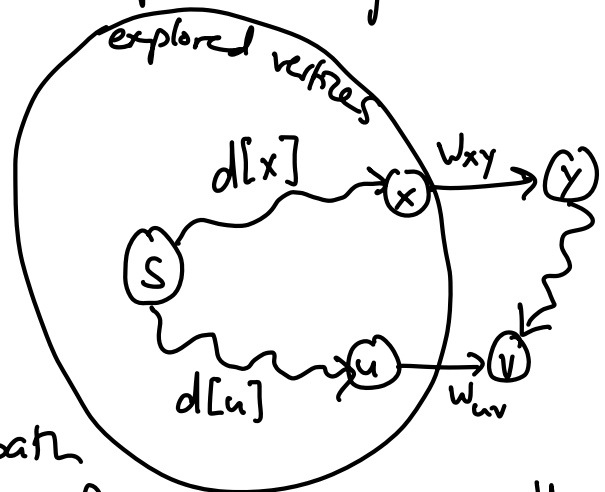
IH: Suppose lemma holds when k vertices are explored.

Let v be the $(k+1)$ st explored vertex;
let $pr[v] = u$.

By IH, the algorithm has already found the shortest path from s to u .

Suppose (for a contradiction) that the shortest path from s to v does not go through u ;
let x be the last explored vertex on the path
and let y be its unexplored neighbor on this path.

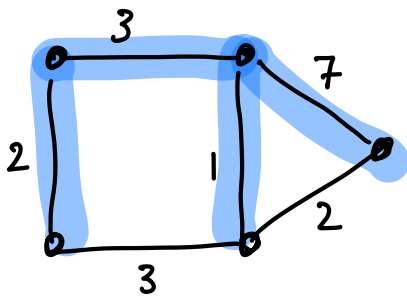
Then $dist(s, v) < d[v]$
 $\leq d[y]$
 $= \text{length of a path from } s \text{ to } y \text{ found by the algorithm}$
 $\leq dist(s, v), \text{ a contradiction}$ □



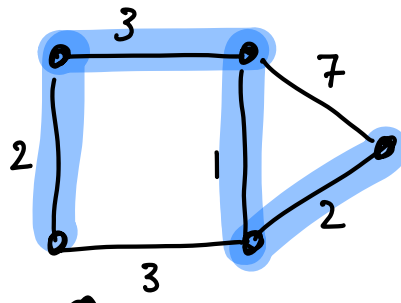
Minimum spanning tree

Goal: Given graph G with edge weights, find a spanning tree T that minimizes the total weight of edges in T .

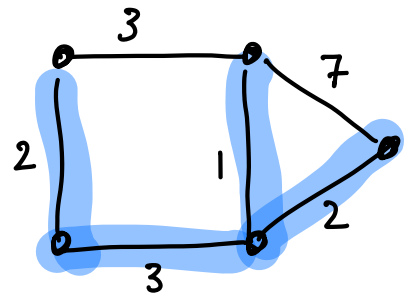
Ex:



cost 13



cost 8



- Assignment 2 is out; due Oct 2
- Greedy algorithms
 - Minimum spanning tree
 - Scheduling
- Up next: Divide and conquer

Sep 19

Def: A cut in a graph is a bipartition of the vertices. ($V = A \cup B$, $A \cap B = \emptyset$)

A cut is nontrivial if neither A nor B is empty.
An edge crosses the cut if it has one vertex on each side.

Lemma: Assume all edge weights are distinct.

Let (A, B) be a nontrivial cut;

let e be the minimum-weight edge crossing this cut.

Then any minimum spanning tree must contain e .

Proof: Let T' be a min. spanning tree that does not contain $e = \{u, v\}$.

Since T' is spanning, there is a (unique) path in T' from u to v .

Since u and v are on opposite sides of the cut, there must be an edge along this path that crosses the cut; call it e' .

Let T be the graph obtained from T' by deleting e' and adding e . (Can show that T is a spanning tree.)

Since e is the min-weight edge crossing the cut, $w_e < w_{e'}$, and since all other edges
weight of edge e

are unchanged, T has lower total weight than T' . $\square$

Kruskal: Add the lowest-cost edge that doesn't create a cycle.

Theorem: Kruskal's alg. outputs a min. spanning tree.

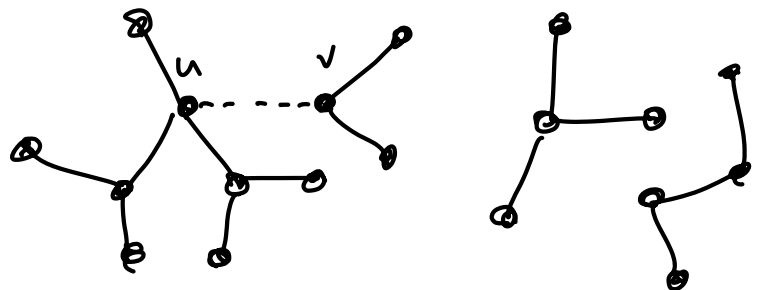
Proof: Suppose the algorithm adds edge $e = \{u, v\}$ to the forest F .

Consider the cut with $A =$ vertices in the component of u in F .

Vertex v is on the other side of the cut since adding e doesn't create a cycle.

There are no edges in F crossing the cut, so e is in fact the min-weight edge crossing the cut.

So by the lemma, every MST contains e . $\square$



Prim: Choose any root and grow a tree by repeatedly adding the non-tree vertex with the lowest attachment cost.

Theorem: Prim's algorithm outputs a MST.

Proof: Suppose the algorithm adds $e = \{u, v\}$ to the forest F . Consider the cut with $A =$ the component of the root in F .

Clearly e crosses the cut, and it has the lowest weight of any such edge by the greedy rule, so by the lemma, every MST contains e .

The algorithm outputs a spanning tree since as long as F is not connected, we can add some edge and still have no cycles. $\square$

These algorithms can both be implemented in time $O(m \log n)$ with appropriate data structures.

Scheduling

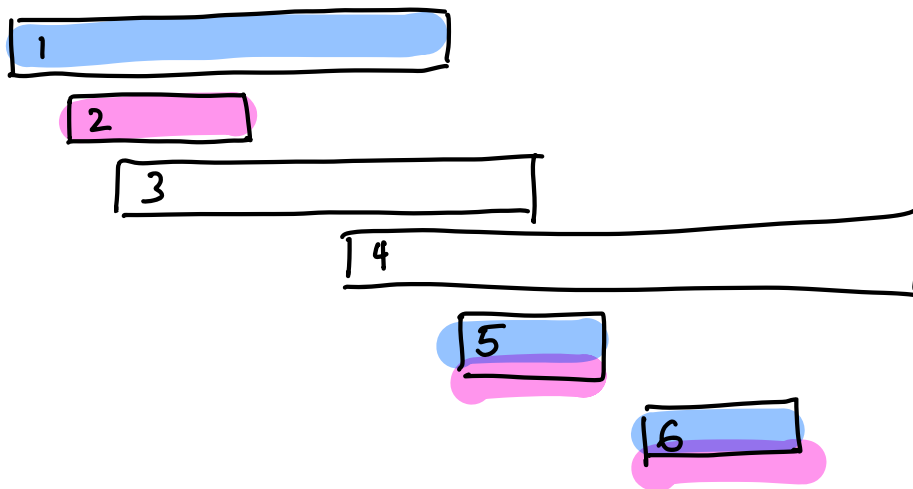
Interval scheduling:

Given n requests with starting times s_i
and finishing times f_i
for $i \in \{1, 2, \dots, n\}$.

A subset of requests is compatible if no two overlap
in time.

Problem: Find a largest possible compatible subset.

Ex:



How to be greedy?

- fewest overlaps $\leftarrow$ not always optimal; see book for counterexample

- shortest activity first



- earliest-finishing activity first $\leftarrow$ optimal!

- earliest activity first



Greedy Interval Schedule (s, f) :

sort tasks by increasing order of finish time

let A be an empty set

let $f_{\text{prev}} = -\infty$

for $i=1$ to n

if $s_i > f_{\text{prev}}$

add task i to A

$f_{\text{prev}} = f_i$

end:if

end:if

→ running time $O(n \log n)$

- Scheduling

- Interval scheduling

- Interval partitioning

- Scheduling to minimize lateness

- Divide and conquer

Sep 24

- Schedule contains no conflicts because we only add intervals that start after the last finished interval.

Theorem: Greedy Interval Schedule outputs an optimal solution.

Proof: Let $G = (g_1, \dots, g_k)$ be the greedy schedule
(with indices of intervals in order of increasing finish time).

Let $B = (b_1, \dots, b_e)$ be an optimal schedule.

Let j be the first index where the schedules differ, so

$$G = (g_1, \dots, g_{j-1}, g_j, \dots, g_k)$$

$$B = (g_1, \dots, g_{j-1}, b_j, \dots, b_e)$$

By the greedy rule, $f_{g_j} \leq f_{b_j}$.

So let's switch B to

$$B' = (g_1, \dots, g_{j-1}, g_j, b_{j+1}, \dots, b_e)$$

This is still a valid schedule, and it has as many intervals as B .

Repeating, we can get another ^{valid} schedule

$$(g_1, \dots, g_k, b_{k+1}, \dots, b_e).$$

Since the greedy algorithm stopped with only k jobs, there cannot be a job b_{k+1} that it could have scheduled, so in fact $k=l$ and the greedy schedule is optimal. $\square$

Interval Positioning

Now suppose we need to schedule all the intervals and want to minimize the resources used.



Def: The depth of a set of intervals is

$$\max_t |\{i \in \{1, \dots, n\} : t \in [s_i, f_i]\}|.$$

Clearly the # of resources is at least the depth.

Greedy Interval Partition (s, f) :

Sort the intervals by starting time

For $i=1$ to n

Let $E = \emptyset$

For $j=1$ to $i-1$

If $[s_j, f_j]$ overlaps $[s_i, f_i]$ then add
color $[j]$ to E

Endfor

Let color $[i]$ be the smallest positive integer that
is not in E

Endfor

Theorem: Using Greedy Interval Partition, each interval is assigned a color from $\{1, 2, \dots, \text{depth}\}$ and no two overlapping intervals are assigned the same color.

Proof: Whenever we assign a color, it is different from the colors of all overlapping intervals, so we never assign the same color to two overlapping intervals.

Now need to show that we don't exclude all colors from $\{1, 2, \dots, \text{depth}\}$.

Say that when considering interval i , it overlaps t previous intervals.

Then $\exists t+1$ overlapping intervals, so $\text{depth} \geq t+1$.

$\Rightarrow t \leq \text{depth} - 1$, so there is always a color available. $\square$

Scheduling to minimize lateness

Setting: each task has a duration t_i and a deadline d_i ($i \in \{1, \dots, n\}$)

define the lateness of task i , if scheduled in $[s_i, f_i]$ as $l_i = \max \{0, f_i - d_i\}$ $s_i + t_i$

goal: minimize $\max_i l_i$ (worst-case lateness)

Greedy algorithms: • schedule in order of increasing deadline d_i

• increasing duration t_i $\times$

$$t_1 = 1, d_1 = 100; \quad t_2 = 10, d_2 = 10$$

• increasing $d_i - t_i$ $\times$

Def: A schedule has an inversion if job i comes before job j
but $d_i > d_j$.

Lemma: All schedules with no inversions (& no idle time)
have the same max lateness.

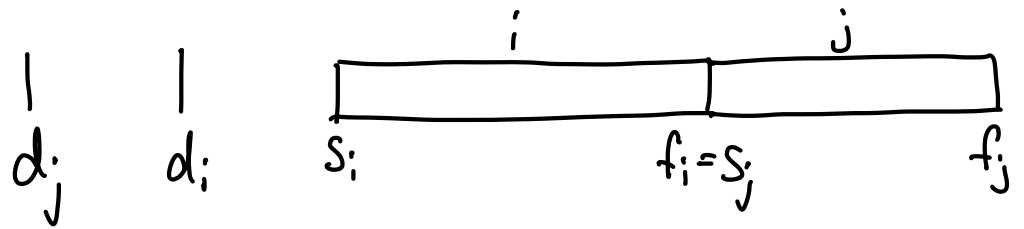
Proof: The only flexibility in such a schedule is in
the order of jobs with the same deadline.
Any such jobs must be scheduled consecutively.
The finishing time of the last of these jobs doesn't
depend on the order. $\square$

Lemma: There is an optimal schedule with no inversions
(& no idle time).

Proof: We show how to convert an optimal with inversions
into one with none, without increasing the max
lateness.

If there is an inversion, then there must be
jobs i, j that are adjacent (with i before j)
with $d_i > d_j$.

Swapping i and j gives a schedule with one less inversion. We claim it has no worse lateness.



• Scheduling to minimize lateness

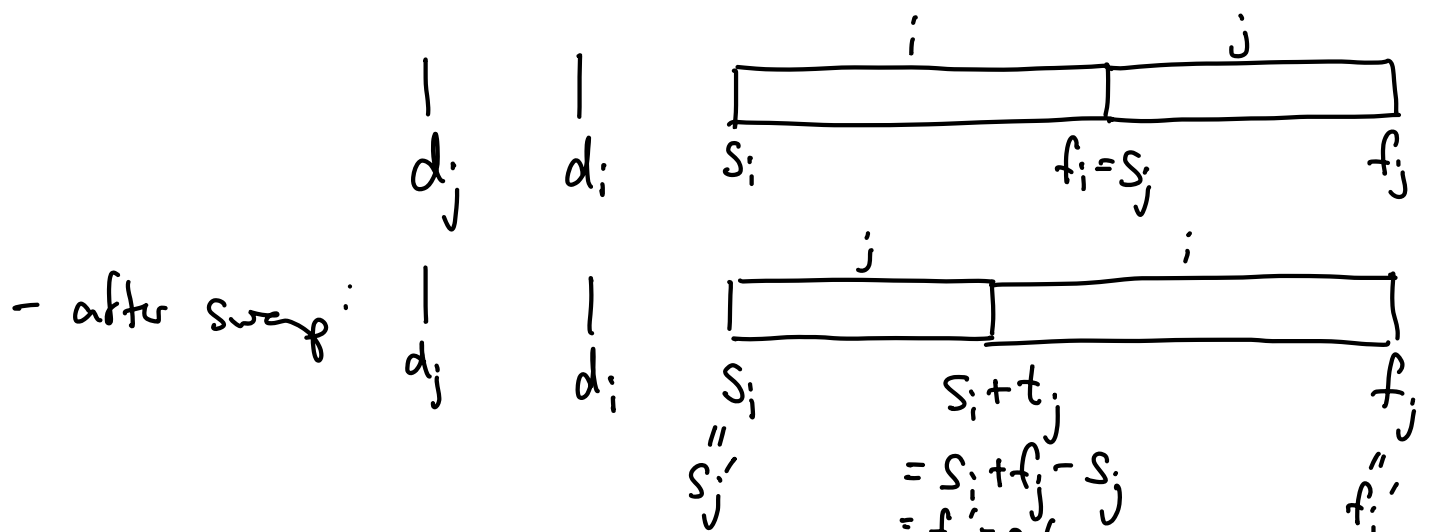
Sep 26

• Divide and conquer

- Closest points

- Polynomial multiplication and the fast Fourier transform

- before swap, lateness is $\max \{ f_i - d_i, f_j - d_j \}$
 $= f_j - d_j$



$$\text{lateness is } \max \left\{ \underbrace{f_j - d_i}_{\leq f_j - d_j \text{ since } d_i \geq d_j}, \underbrace{s_i + f_j - s_j - d_j}_{= (f_j - d_j) - (s_j - s_i) \leq f_j - d_j \text{ since } s_j \geq s_i} \right\}$$

so the lateness is no larger after the swap.



Divide and conquer

Ex: Mergesort : To sort a list of length $n \dots$

- divide list into two halves and sort each of them (recursively)
- merge the two lists in time $O(n)$

let $T(n)$ = running time on instances of length n

$$T(n) = 2T(n/2) + O(n) \quad T(1) = O(1)$$

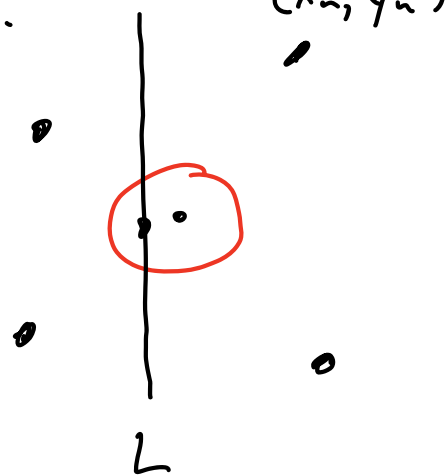
$$\Rightarrow T(n) = O(n \log n)$$

Closest points

Problem: Given n points in the plane.
Find the closest pair.

$(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$

Simple approach: run over all pairs.
time $O(n^2)$



Simpler version: n points in 1D. $x_1, x_2, \dots, x_n$

time $O(n \log n)$ by sorting

Main idea: Recursively divide into left & right halves

Find the closest pair of points on each side

Check whether there is a closer pair of points
with one on left & one on right

As we recurse, maintain sorted lists by x & y coordinates

Notation: Input points $P = \{p_1, \dots, p_n\}$ $p_i = (x_i, y_i)$

$Q =$ points in P with the first $\lceil n/2 \rceil$ x coordinates

$R =$ points in P with the last $\lfloor n/2 \rfloor$ x coordinates

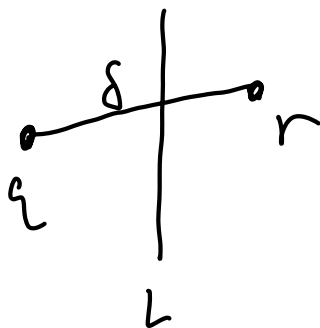
Find the closest pair of points in Q (q_0^*, q_1^*)
and R (r_0^*, r_1^*)

let $\delta = \min \{ \text{dist}(q_0^*, q_1^*), \text{dist}(r_0^*, r_1^*) \}$

let x^* be the largest x coordinate of a point in Q

let L be the line $x = x^*$

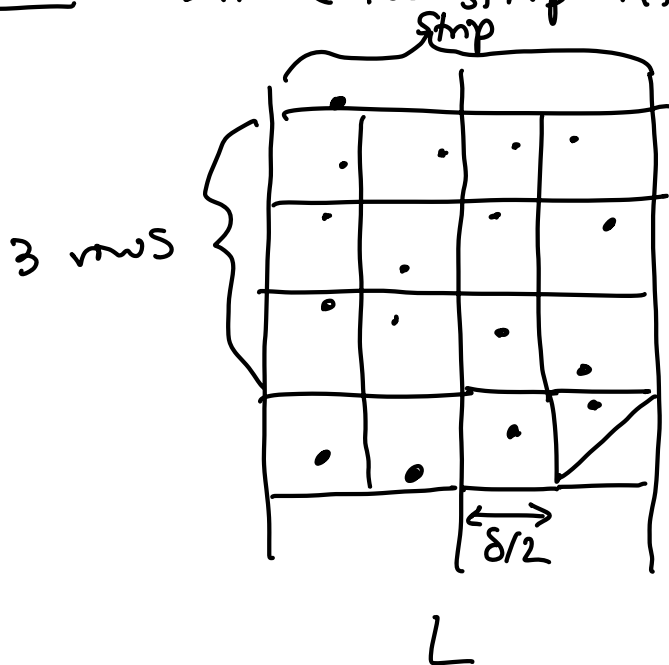
Lemma: If $\exists q \in Q, r \in R$ with $\text{dist}(q, r) < \delta$
 then both q, r must be within distance δ of L .



Let S be the strip of width 2δ centered on L .

Lemma: If two points in S have distance less than δ ,
 then they must be within 15 positions of each
 other in the list of points sorted by y coordinate.

Proof: Divide the strip into squares of side length $\frac{\delta}{2}$:



Claim: We cannot have two
 points in a box.

If there were, they would
 be on the same side and
 have distance at most
 $\sqrt{2} \frac{\delta}{2} = \frac{\delta}{\sqrt{2}}$

If two points in S differ by 16 or more positions in the y -sorted list, then they differ by at least 3 rows of boxes, so they have distance at least $3\frac{\delta}{2} > \delta$. $\square$

- Assignment 2 due tomorrow
- Midterm exam in class on Oct. 8.
- Midterm practice problems on Canvas
- Divide and conquer:
 - Closest points
 - Polynomial multiplication / FFT
- Up next: Dynamic programming

Oct 1

Running time for closest points:

Initial sorting: $O(n \log n)$

For the main algorithm, let the running time be $T(n)$.

$$T(n) = 2T(n/2) + O(n)$$



$$T(n) = O(n \log n)$$

↑ selecting the points

looking through the points to see if there is a closer pair.

Polynomial multiplication

Problem: given polynomials $a(x) = a_0 + a_1x + a_2x^2 + \dots + a_{n-1}x^{n-1}$
and $b(x) = b_0 + b_1x + b_2x^2 + \dots + b_{n-1}x^{n-1}$

compute their product

$$\begin{aligned}c(x) &= a(x)b(x) \\&= \left(\sum_{j=0}^{n-1} a_j x^j \right) \left(\sum_{k=0}^{n-1} b_k x^k \right) \\&= \sum_{j=0}^{n-1} \sum_{k=0}^{n-1} a_j b_k x^{j+k} \\&= \sum_{l=0}^{2n-2} \sum_{j=0}^l a_j b_{l-j} x^l \\&= \sum_{l=0}^{2n-2} c_l x^l\end{aligned}$$

$$c_l = \sum_{j=0}^l a_j b_{l-j} \quad l=0 \text{ to } 2n-2$$

- computing one c_l : time $O(n)$
- computing all c_l s: time $O(n^2)$.

To do better, use polynomial interpolation:

A degree- d polynomial is uniquely specified by its values at any $d+1$ distinct points.

- Strategy:
- ① Evaluate $a(x)$ and $b(x)$ on 2^{n-1} points
 - ② Compute $c(x) = a(x)b(x)$ on these points
 - ③ Reconstruct the coefficients of $c(x)$ from these evaluations.

Main idea: By choosing the points carefully, we can compute $a(x)$ on all of them in time $O(n \log n)$, even though there are 2^{n-1} computations that would each take time $O(n)$ individually.

Assume d is even.

$$\text{Let } a_{\text{even}}(x) = a_0 + a_2 x + a_4 x^2 + \dots + a_{d-2} x^{\frac{d}{2}-1}$$

$$a_{\text{odd}}(x) = a_1 + a_3 x + a_5 x^2 + \dots + a_{d-1} x^{\frac{d}{2}-1}$$

$$\text{Then } a(x) = a_{\text{even}}(x^2) + x a_{\text{odd}}(x^2)$$

$$\text{Ex: } d=4 \quad a_{\text{even}}(x) = a_0 + a_2 x$$

$$a_{\text{odd}}(x) = a_1 + a_3 x$$

$$\begin{aligned} a_{\text{even}}(x^2) + x a_{\text{odd}}(x^2) &= a_0 + a_2 x^2 + x(a_1 + a_3 x^2) \\ &= a_0 + a_1 x + a_2 x^2 + a_3 x^3 \\ &= a(x) \end{aligned}$$

Choose the points so that the set of squares of the points is half as big.

$$\text{Ex: } \{+1, -1\} \xrightarrow{\text{square}} \{+1\}$$

$$\begin{array}{ccc} \{+1, -1, +2, -2\} & \longrightarrow & \{+1, +4\} \\ & \not\rightarrow & \{1, 16\} \end{array}$$

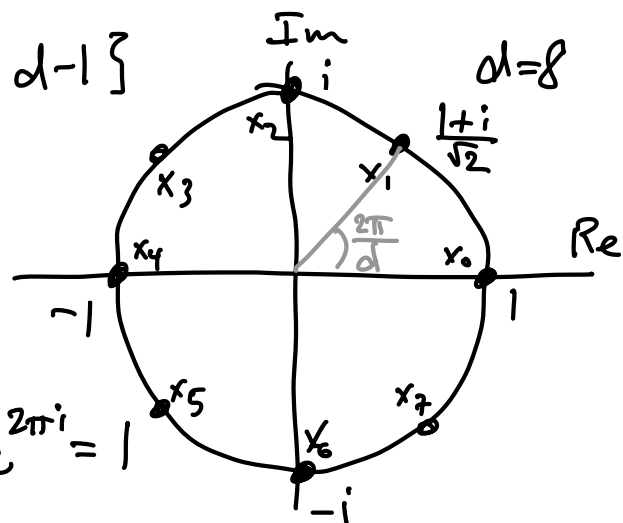
$$\begin{array}{ccc} \text{Ex: } \{1, -1, i, -i\} & \longrightarrow & \{1, -1\} \\ & \longrightarrow & \{1\} \end{array}$$

In general: take d^{th} roots of unity, $f(\theta) = e^{i\theta}$

Def: $x_j = \omega_d^j$ for $j \in \{0, 1, \dots, d-1\}$
 with $\omega_d = e^{2\pi i/d}$
 $= \exp(2\pi i/d)$

$$\begin{aligned} x_j^2 &= \omega_d^{2j} \\ &= \omega_d^{2j \bmod d} \\ &= x_{2j \bmod d} \end{aligned}$$

since $\omega_d^d = e^{2\pi i} = 1$



$$\begin{aligned} \left(\frac{1+i}{\sqrt{2}}\right)^2 &= \frac{1}{2}(1+i)^2 \\ &= \frac{1}{2}(1+2i+i^2) = i \end{aligned}$$

Cost of evaluating $a(x)$ at x_j for $j \in \{0, 1, \dots, d-1\}$:

$$T(d) = 2T(d/2) + O(d) \rightarrow T(d) = O(d \log d)$$

Remaining step ③: recover coefficients of $c(x)$ from the evaluations $c(\omega_d^j)$

$$a(x) = a_0 + a_1 x + a_2 x^2 + \dots + a_{d-1} x^{d-1}$$

$$= (1 \quad x \quad x^2 \quad \dots \quad x^{d-1}) \begin{pmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_{d-1} \end{pmatrix}$$

$$\begin{pmatrix} a(\omega_d^0) \\ a(\omega_d^1) \\ a(\omega_d^2) \\ \vdots \\ a(\omega_d^{d-1}) \end{pmatrix} = \begin{pmatrix} 1 & \omega_d^0 & \omega_d^{0 \cdot 2} & \dots & \omega_d^{0 \cdot (d-1)} \\ 1 & \omega_d^1 & \omega_d^{1 \cdot 2} & \dots & \omega_d^{1 \cdot (d-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega_d^{d-1} & (\omega_d^{d-1})^2 & \dots & (\omega_d^{d-1})^{d-1} \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_{d-1} \end{pmatrix}$$

• Midterm exam in class on Oct. 8

Oct 3

Solve some practice problems!

• Polynomial multiplication and the FFT

• Dynamic programming: Weighted interval scheduling

$$\begin{pmatrix} a(\omega_d^0) \\ a(\omega_d^1) \\ a(\omega_d^2) \\ \vdots \\ a(\omega_d^{d-1}) \end{pmatrix} = \sqrt{d} F \begin{pmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_{d-1} \end{pmatrix} \quad \text{where } \omega_d = \exp(2\pi i/d)$$

$F_{jk} = \frac{1}{\sqrt{d}} \omega_d^{jk}$
 $\uparrow$
 discrete Fourier transform

$$\begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_{d-1} \end{pmatrix} = (\sqrt{d} F)^{-1} \begin{pmatrix} a(\omega_d^0) \\ a(\omega_d^1) \\ \vdots \\ a(\omega_d^{d-1}) \end{pmatrix}$$

It turns out that $\sqrt{d} F$ is a unitary matrix:

$$(\sqrt{d} F)^{-1} = ((\sqrt{d} F)^*)^T$$

$\nwarrow$ complex conjugate
 $\nwarrow$ transpose

$$\omega_d = e^{2\pi i/d}$$

$$\omega_d^{-1} = e^{-2\pi i/d}$$

$$(\sqrt{d} F)^{-1}_{jk} = \frac{1}{\sqrt{d}} \omega_d^{-jk} = \frac{1}{\sqrt{d}} \exp(-2\pi jk/d)$$

So there is an $O(n \log n)$ time algorithm for the polynomial multiplication problem!

Dynamic programming

Ex: Weighted interval scheduling

Given intervals $[s_i, f_i]$ for $i \in \{1, \dots, n\}$,

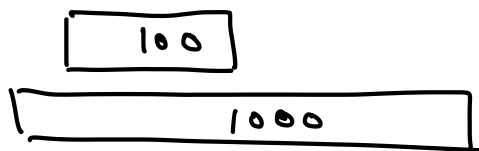
find a ~~largest-possible subset~~ of non-overlapping intervals.

Modification: interval i has a value $v_i \in \mathbb{R}$

Goal: find a subset S of non-overlapping intervals

that maximizes $\sum_{i \in S} v_i$

Earliest-finishy first is no longer optimal:

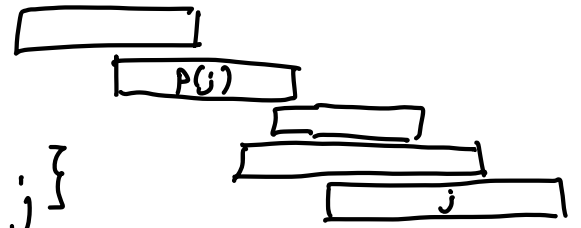


Sort intervals in order of finishing times: $f_1 \leq f_2 \leq \dots \leq f_n$

Work backward from the end. Is interval n part of the optimal solution?

- no: solution is on intervals $\{1, 2, \dots, n-1\}$
- yes: optimal solution is interval n + optimal solution on intervals $\{1, 2, \dots, p(n)\}$, where $p(j) = \max \{i < j : \text{intervals } i \text{ and } j \text{ are disjoint}\}$

In general, let $\text{opt}(j)$ be the optimal value using intervals $\{1, \dots, j\}$



$$\text{opt}(j) = \max \{ \text{opt}(j-1), \text{opt}(p(j)) + v_j \}$$

and interval j is part of the optimal schedule if $\text{opt}(p(j)) + v_j \geq \text{opt}(j-1)$

Compute $\text{Opt}(j)$:

if $j=0$ then

return 0

else

return $\max \{ \text{ComputeOpt}(j-1), v_j + \text{ComputeOpt}(p(j)) \}$

end:if

Running time: let $T(j)$ = running time of $\text{ComputeOpt}(j)$

$$T(j) = T(j-1) + T(p(j)) + O(1)$$

↑ could be $j-1$ in the worst case

$$T(j) = 2T(j-1) + O(1)$$

→ grows like 2^j

But this is very wasteful. We can save a lot of effort by storing the value of $\text{ComputeOpt}(j)$ and not recomputing it.

Store a "memo" that records these values.

Initially, let $M[j] = \emptyset$ for all $j \in \{1, \dots, n\}$

$M\text{ComputeOpt}(j)$:

if $j=0$ then

return 0

elseif $M[j] \neq \emptyset$ then

return $M[j]$

else

let $M[j] = \max \{ M\text{ComputeOpt}(j-1), \\ v_j + M\text{ComputeOpt}(p(j)) \}$

return $M[j]$

endif